作业四

一、 并行化时我们需要注意并识别循环依赖，分析以下两个 C 语言循环片段，判断其是否可以直接使用 #pragma omp parallel for 进行并行化。如果不能，请说明原因：

片段 A：

```
for (i = 1; i < n; i++) {
    A[i] = A[i-1] + B[i];
}
```

片段 B：

```
int index = 0;
for (i = 0; i < n; i++) {
    index = (i + offset) % n; //offset 为一个固定的常数
    C[index] = A[i] * B[i];
}
```

二、请阅读以下使用 OpenMP 并行化的 C 语言代码片段，假设 n 非常大，线程数为 4 个：

```
double result = 0.0;
// 使用 OpenMP 并行化
#pragma omp parallel for reduction(+: result)
for (int i = 0; i < n; i++) {
    result += HeavyCompute(i);
}
```

其中 HeavyCompute(i) 是一个模拟函数，i 越大，计算所需的时间越长

（1） 虽然这段代码计算结果是正确的，但在性能上可能会出现什么问题？

（2） OpenMP 的 parallel for 默认使用哪种调度策略？为什么这种策略在上述场景下效率低下？

（3） 请修改 #pragma 指令，使用一种更合适的调度策略来平衡负载，提高程序运行速度。

三、请阅读以下代码片段并回答问题

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, s = 0, a = 5;
    #pragma omp parallel num_threads(3) private(i) shared(a)
reduction(+:s)
    {
        int tid = omp_get_thread_num();
        a += tid;
        #pragma omp for
        for (i = 1; i <= 6; i++) {
            s += i;
        }
    }
    printf("s = %d\n", s);
    printf("a = %d\n", a);
    return 0;
}
```

（1） 变量 i, s, a 分别是什么变量？

（2） 指出这段代码所存在的问题。

（3） 分析最终 a 和 s 的输出。

四、某程序使用 OpenMP 并行化如下循环：

```
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 16; i++) {
    work(i);
}
```

设系统采用默认调度方式，并假设每次迭代 work(i) 的执行时间基本相同。

请回答下列问题：

（1）若 16 次迭代被连续地、尽可能均匀地分配给 4 个线程，请写出每个线程可能负责的迭代编号范围。

（2）这种分配方式有什么优点？

（3）如果不同迭代的计算量差异非常大，那么默认这种分配方式可能会带来什么问题？

五、阅读下面的 OpenMP 程序片段：

```
double x = 0.0;
double y = 0.0;

#pragma omp parallel num_threads(4)
{
    for (int i = 0; i < 1000; i++) {
        #pragma omp atomic
        x += 1.0;

        #pragma omp atomic
        y += 1.0;
    }
}
```

请回答下列问题：

（1）这两条 atomic 指令分别保护的是什么操作？

（2）对变量 x 的更新和对变量 y 的更新，是否会被视为同一个“临界区”？为什么？

（3）如果把两条 atomic 改成一个 critical 代码块，将两次加法都放进去，程序的并发性会发生什么变化？